

Lab 9

Latches & Flip-flops



课程名称：数字逻辑电路设计

作者：王传宇 3250102681

专业：计算机科学与技术

邮箱：3250102681@zju.edu.cn

时间：2026-5-7

地点：紫金港东四 509 室

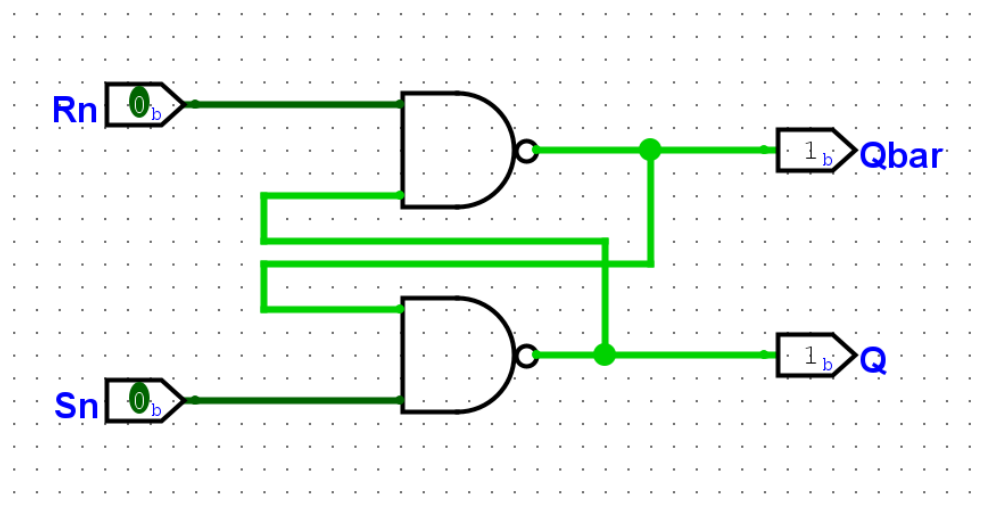
第一部分：操作方法与实验步骤

本次实验使用 Logisim-evolution 完成各类锁存器与触发器的原理图设计，并导出为 Verilog 文件，在 Vivado 中编写 testbench 进行功能仿真。

实验中所有锁存器与触发器均采用 NAND 门实现。

SR 锁存器

使用两个交叉耦合的 NAND 门构成 SR 锁存器，其中输入为低电平有效。



对应仿真文件如下：

```
`timescale 1ns / 1ps

module SRNANDLatch_tb;

    reg R;
    reg S;

    wire Q;
    wire Qbar;

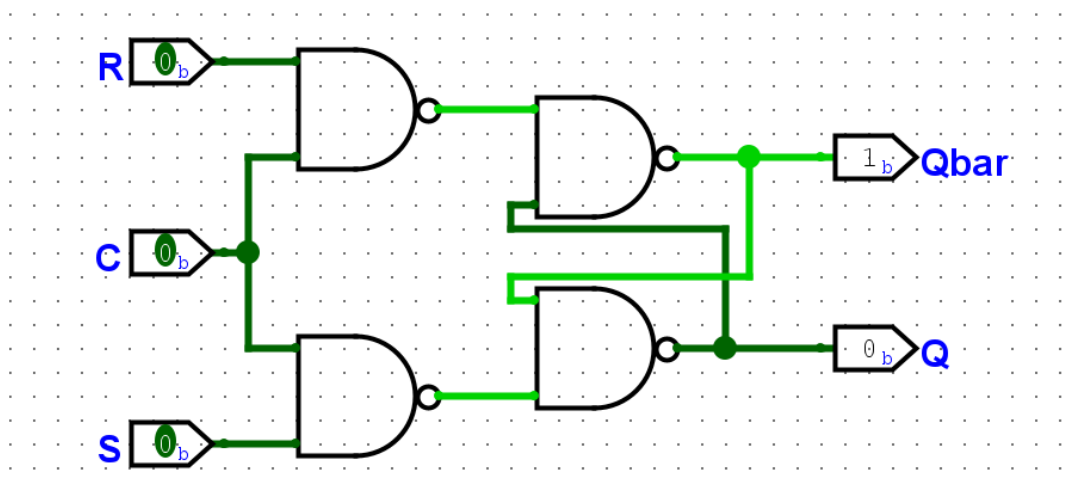
    main uut (
        .R(R),
        .S(S),
        .Q(Q),
        .Qbar(Qbar)
    );

    initial begin
```

```
R = 1;  
S = 1;  
  
#10;  
S = 0;  
R = 1;  
  
#10;  
S = 1;  
  
#20;  
  
R = 0;  
S = 1;  
  
#10;  
R = 1;  
  
#20;  
  
$finish;  
  
end  
  
endmodule
```

门控 SR 锁存器

在 SR 锁存器基础上增加使能端 C，仅当 C 为高电平时输入信号才会影响输出。



对应仿真文件如下:

```
`timescale 1ns / 1ps

module GatedSRLatch_tb;

    reg R;
    reg S;
    reg C;

    wire Q;
    wire Qbar;

    main uut (
        .R(R),
        .S(S),
        .C(C),
        .Q(Q),
        .Qbar(Qbar)
    );

    initial begin

        // =====
        // C = 0 : latch disabled
        // =====

        C = 1'b0;

        // S=1 R=1 : hold
        S = 1'b1;
        R = 1'b1;
        #20;

        // S=0 R=1 : should NOT set
        S = 1'b0;
        R = 1'b1;
        #20;

        // S=1 R=0 : should NOT reset
        S = 1'b1;
        R = 1'b0;
        #20;

        // S=0 R=0 : should still have no effect
```

```
S = 1'b0;
R = 1'b0;
#20;

// =====
// C = 1 : latch enabled
// =====

C = 1'b1;

// S=1 R=1 : hold
S = 1'b1;
R = 1'b1;
#20;

// S=0 R=1 : SET
S = 1'b0;
R = 1'b1;
#20;

// S=1 R=1 : hold
S = 1'b1;
R = 1'b1;
#20;

// S=1 R=0 : RESET
S = 1'b1;
R = 1'b0;
#20;

// S=1 R=1 : hold
S = 1'b1;
R = 1'b1;
#20;

// S=0 R=0 : INVALID STATE
// placed at the end intentionally

S = 1'b0;
R = 1'b0;
#20;

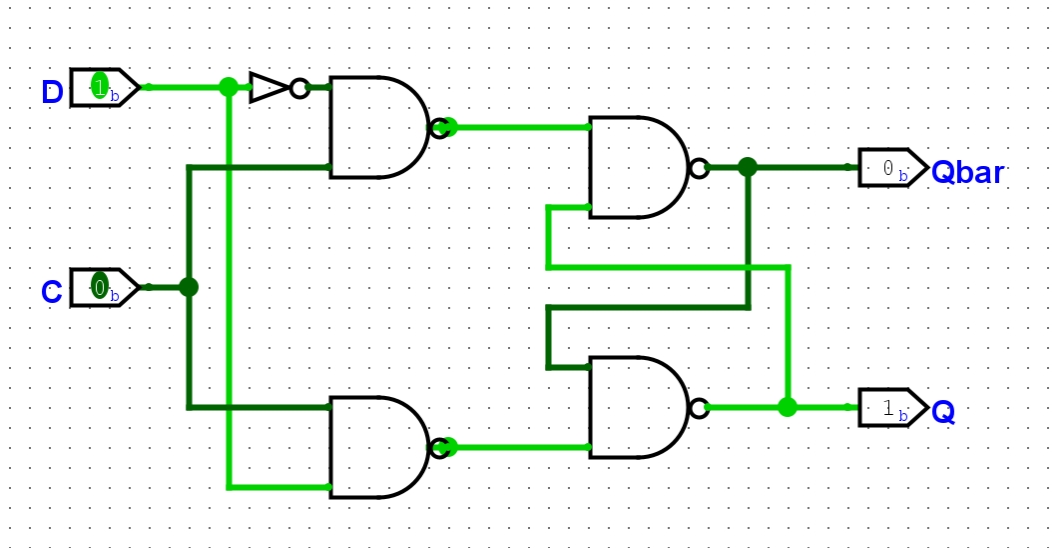
$finish;
```

```
end
```

```
endmodule
```

门控 D 锁存器

使用 NAND 门实现门控 D 锁存器，并观察其“空翻”现象。



对应仿真文件如下：

```
`timescale 1ns / 1ps

module GatedDLatch_tb;

    reg D;
    reg C;

    wire Q;
    wire Qbar;

    GatedDLatch uut (
        .D(D),
        .C(C),
        .Q(Q),
        .Qbar(Qbar)
    );

    initial begin

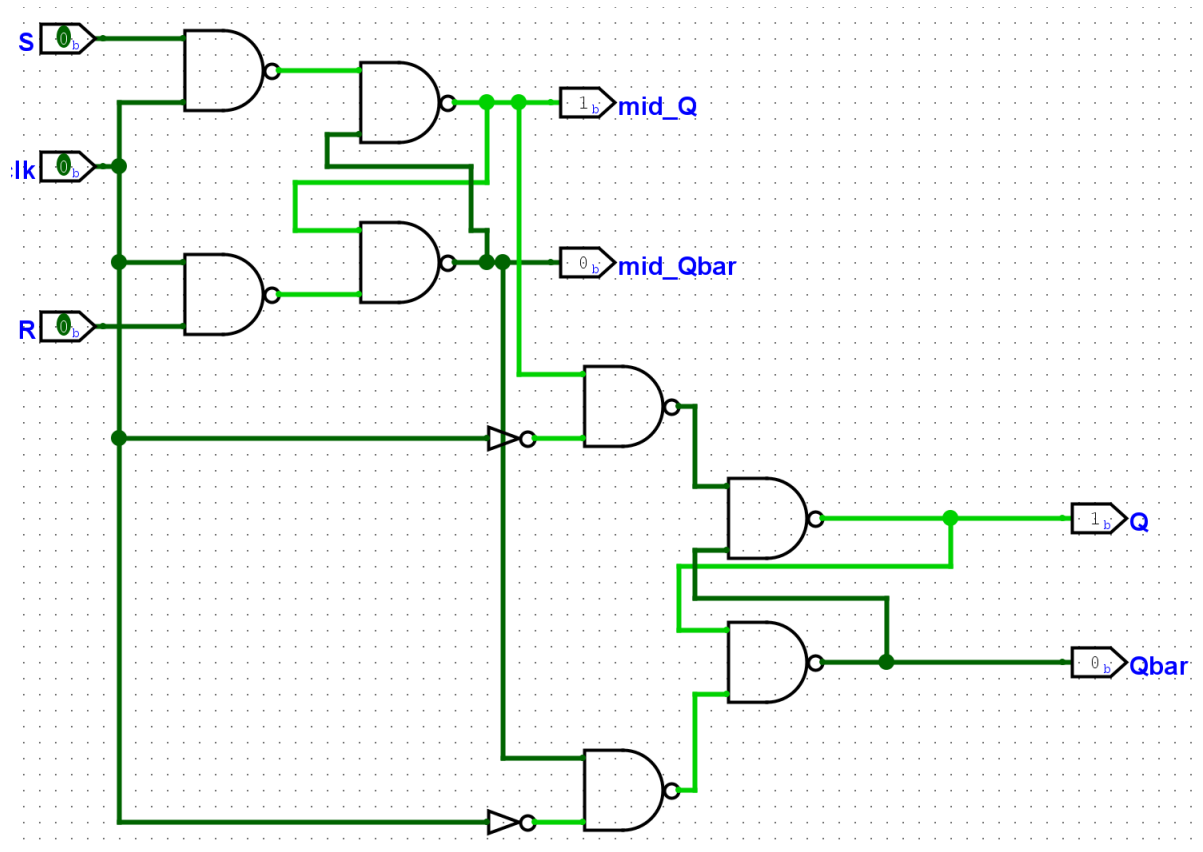
        // =====
        // Initialize latch to a stable state
```

```
// =====  
  
C = 1'b1;  
D = 1'b0;  
#20;  
  
// =====  
// C = 0 : hold mode  
// D changes should NOT affect Q  
// =====  
  
C = 1'b0;  
  
D = 1'b0;  
#20;  
  
D = 1'b1;  
#20;  
  
D = 1'b0;  
#20;  
  
// =====  
// C = 1 : transparent mode  
// Q follows D immediately  
// This demonstrates race-through behavior  
// =====  
  
C = 1'b1;  
  
D = 1'b1;  
#20;  
  
D = 1'b0;  
#20;  
  
D = 1'b1;  
#20;  
  
D = 1'b0;  
#20;  
  
D = 1'b1;  
#20;
```

```
// =====  
// Disable again  
// Q should hold last value  
// =====  
  
C = 1'b0;  
  
D = 1'b0;  
#20;  
  
D = 1'b1;  
#20;  
  
$finish;  
  
end  
  
endmodule
```

正边沿 **SR** 主从触发器

使用两个门控 **SR** 锁存器构成主从结构，其中主锁存器由 **clk** 控制，从锁存器由反相信号控制。



对应仿真文件如下:

```
`timescale 1ns / 1ps

module MasterSlave_flipFlop_tb;

    reg S;
    reg R;
    reg clk;

    wire mid_Q;
    wire Q;
    wire Qbar;

    main uut (
        .S(S),
        .R(R),
        .clk(clk),
        .mid_Q(mid_Q),
        .Q(Q),
        .Qbar(Qbar)
    );
```

```
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin

    S = 1;
    R = 1;

    #8;
    S = 0;

    #10;
    S = 1;

    #12;
    R = 0;

    #10;
    R = 1;

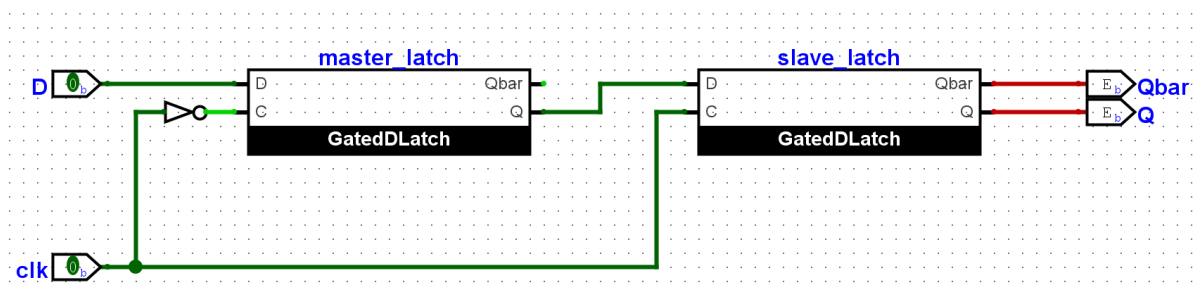
    #20;

    $finish;
end

endmodule
```

正边沿 D 触发器

通过主从结构实现正边沿 D 触发器。



对应仿真文件如下:

```
`timescale 1ns / 1ps

module GatedDLatch_tb;

reg D;
reg C;

wire Q;
wire Qbar;

GatedDLatch uut (
    .D(D),
    .C(C),
    .Q(Q),
    .Qbar(Qbar)
);

initial begin

    // =====
    // Initialize latch to a stable state
    // =====

    C = 1'b1;
    D = 1'b0;
    #20;

    // =====
    // C = 0 : hold mode
    // D changes should NOT affect Q
    // =====

    C = 1'b0;

    D = 1'b0;
    #20;

    D = 1'b1;
    #20;

    D = 1'b0;
    #20;

    // =====
```

```
// C = 1 : transparent mode
// Q follows D immediately
// This demonstrates race-through behavior
// =====

C = 1'b1;

D = 1'b1;
#20;

D = 1'b0;
#20;

D = 1'b1;
#20;

D = 1'b0;
#20;

D = 1'b1;
#20;

// =====
// Disable again
// Q should hold last value
// =====

C = 1'b0;

D = 1'b0;
#20;

D = 1'b1;
#20;

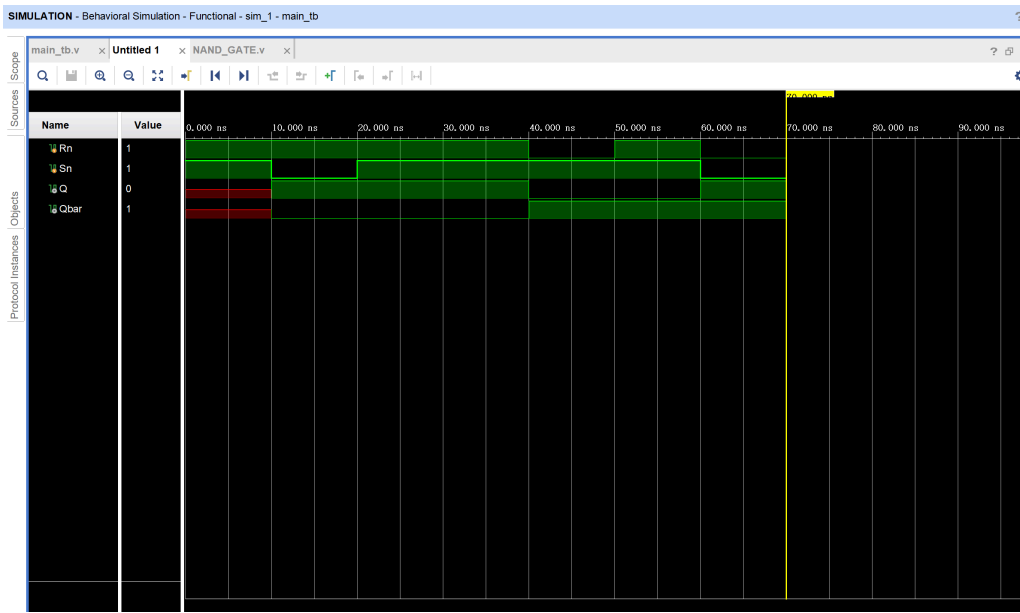
$finish;

end

endmodule
```

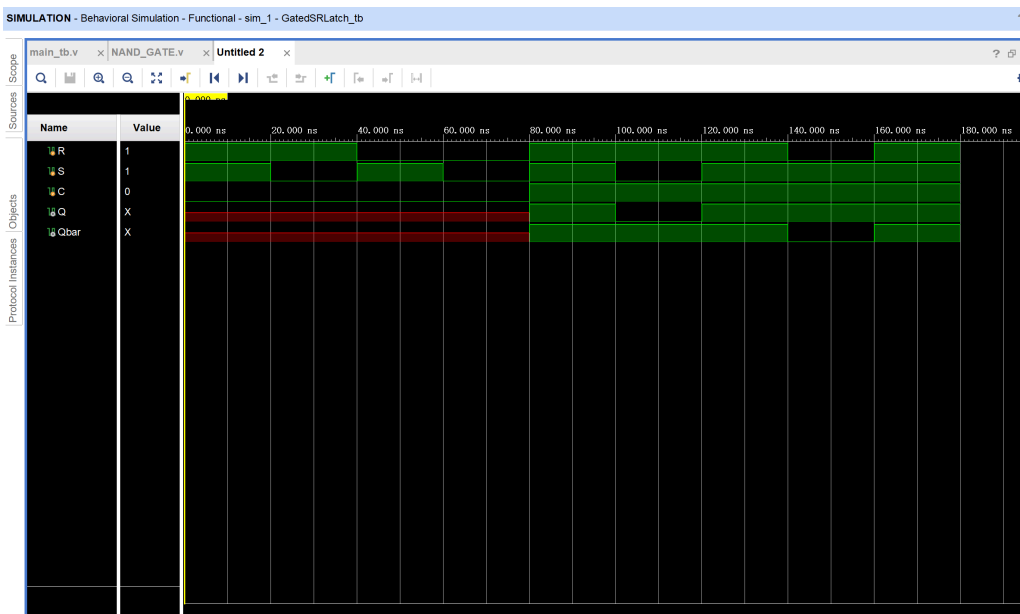
第二部分：实验结果与分析

SR 锁存器波形分析



当 $R=S=1$ 时，锁存器保持原状态不变。当 S 变为低电平时，锁存器被置位， Q 输出为 1， $Qbar$ 输出为 0。随后 S 恢复为高电平后，输出保持不变，体现了锁存器的存储功能。之后令 R 变为低电平，锁存器复位， Q 输出变为 0， $Qbar$ 输出变为 1。

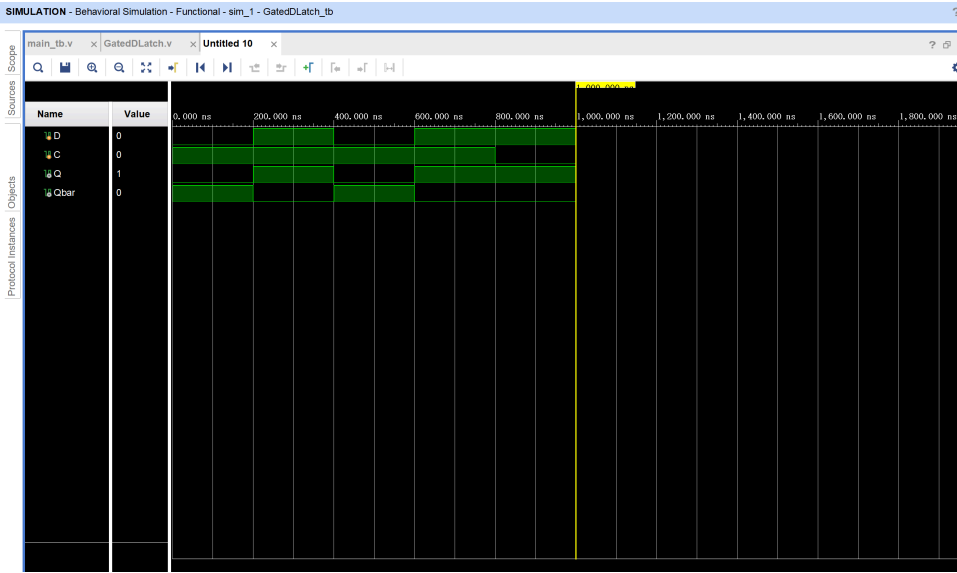
门控 SR 锁存器波形分析



当使能信号 $C=0$ 时，输入 R 、 S 的变化不会影响输出，锁存器保持原状态。当 $C=1$ 时，输入信号开始生效。实验中先通过 $S=0$ 实现置位操作，使 Q 输

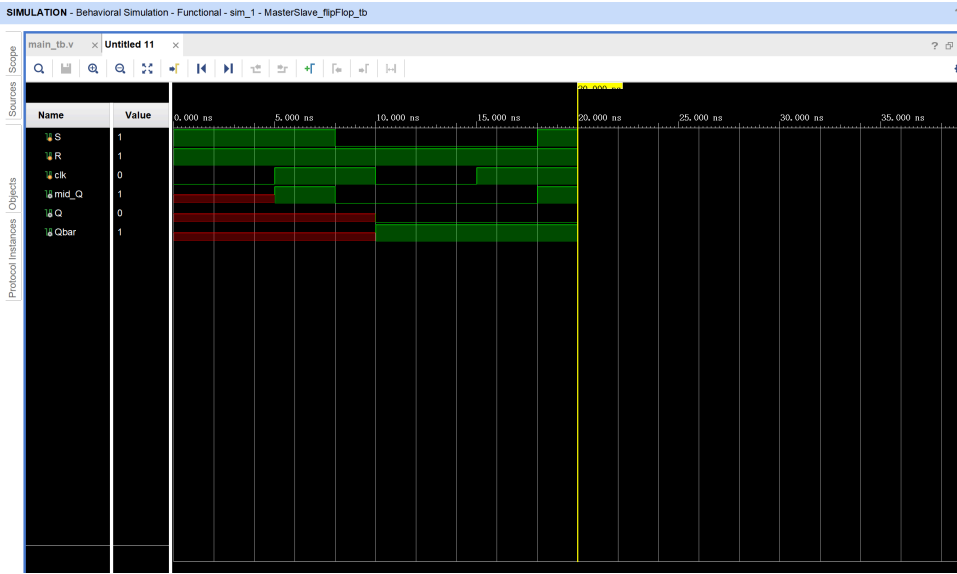
出为 1；随后通过 $R=0$ 实现复位操作，使 Q 输出为 0。实验结果表明门控 SR 锁存器仅在使能有效时响应输入。

门控 D 锁存器波形分析



当 $C=1$ 时，输出 Q 会跟随输入 D 的变化而变化，因此表现出“透明”特性。在实验波形中可以看到 D 多次变化时 Q 同步变化，这体现了门控 D 锁存器的空翻现象。当 $C=0$ 后，锁存器进入保持状态，此时即使 D 继续变化， Q 也不会发生改变。

正边沿 SR 主从触发器波形分析



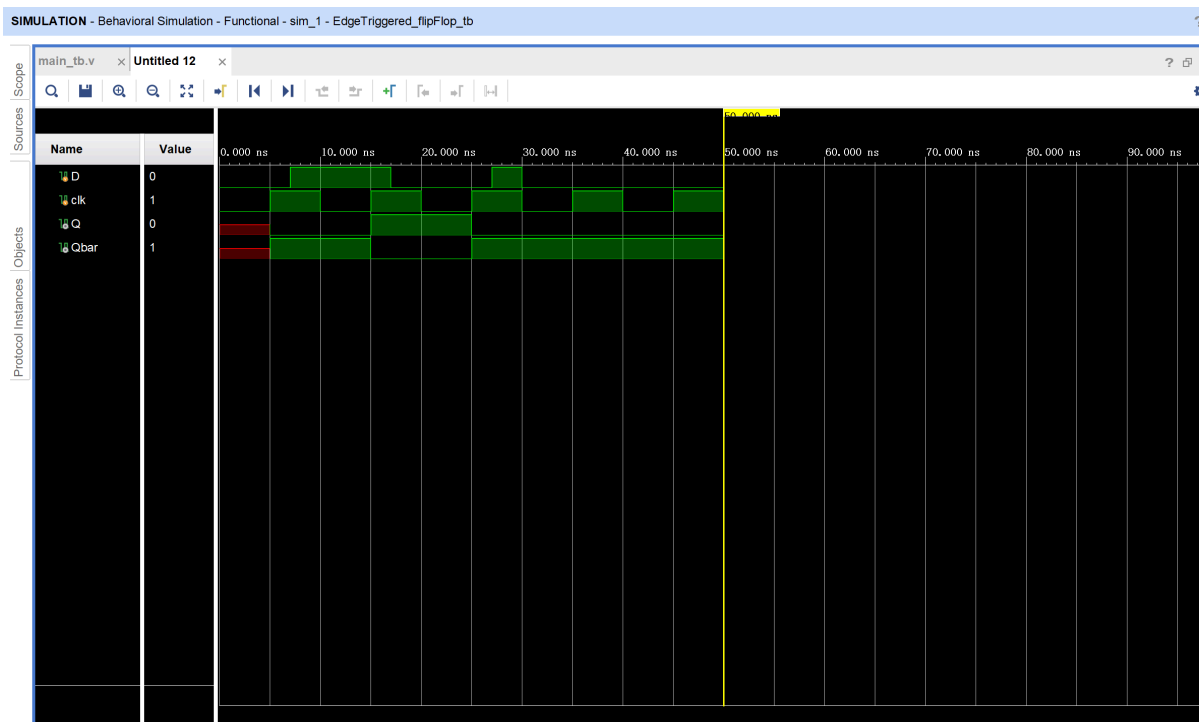
由波形可见，在初始阶段输出存在短暂不定态，这是 NAND 型锁存器反馈结构在初始未定状态下的正常现象。经过若干时钟周期后，电路进入稳定工作状态。

当 $clk=1$ 时，主锁存器处于使能状态，因此 mid_Q 会随着输入 S 、 R 的变化而改变；而从锁存器关闭，因此输出 Q 保持不变。

当时钟从高电平跳变为低电平后，主锁存器关闭，其最终状态被传递到从锁存器，因此 Q 在时钟边沿后发生变化。这说明该结构能够实现主从触发器的边沿触发特性。

从波形中还可以观察到，“一次性采样”现象得到了体现：即使输入信号在时钟有效期间发生变化，最终只有主锁存器在时钟关闭瞬间保存的状态会被传递到输出端，而不会在整个高电平期间持续影响输出 Q 。

正边沿 D 触发器波形分析



D 触发器仅在时钟上升沿采样输入 D 。在两个时钟边沿之间，即使 D 发生变化， Q 也不会立刻改变。只有在时钟出现上升沿时，当前 D 的值才会被锁存到输出 Q 中。实验结果表明该电路具有良好的同步触发特性。

第三部分：讨论与心得

本次实验是第一次较为系统地接触锁存器与触发器的时序逻辑设计。与之前组合逻辑实验相比，本实验最大的特点在于：电路不仅与当前输入有关，还与历史状态有关，因此在调试过程中必须同时关注反馈路径、初始状态以及时钟行为。尤其是在 NAND 型 SR 锁存器与门控 D 锁存器中，由于存在交叉反馈结构，仿真初期经常出现红色或蓝色的不定态波形。起初误以为是连线错误，但在进一步分析后认识到，这实际上是反馈电路在未初始化时的正常现象。通过调整输入激励顺序、增加初始化阶段，并避免非法输入组合，最终使电路能够稳定进入正常工作状态。

在主从触发器部分，调试过程比预期更加复杂。由于 Vivado 在部分非法状态下会陷入不定态传播，导致波形后半段无法继续显示，因此需要重新设计 testbench 的输入顺序，避免在中途过早进入非法输入状态。同时，在观察波形时，也逐渐理解了“主锁存器—从锁存器”结构如何利用时钟实现边沿触发，以及“一次性采样”与“空翻现象”之间的区别。相比单纯阅读理论，亲自搭建电路并观察波形变化后，对时序逻辑的工作机制有了更加直观的认识。这次实验也让我体会到：在数字电路设计中，输入激励的设计与状态分析同样重要，一个看似简单的非法输入就可能导致整个仿真失效，因此在设计与验证时必须更加严谨。